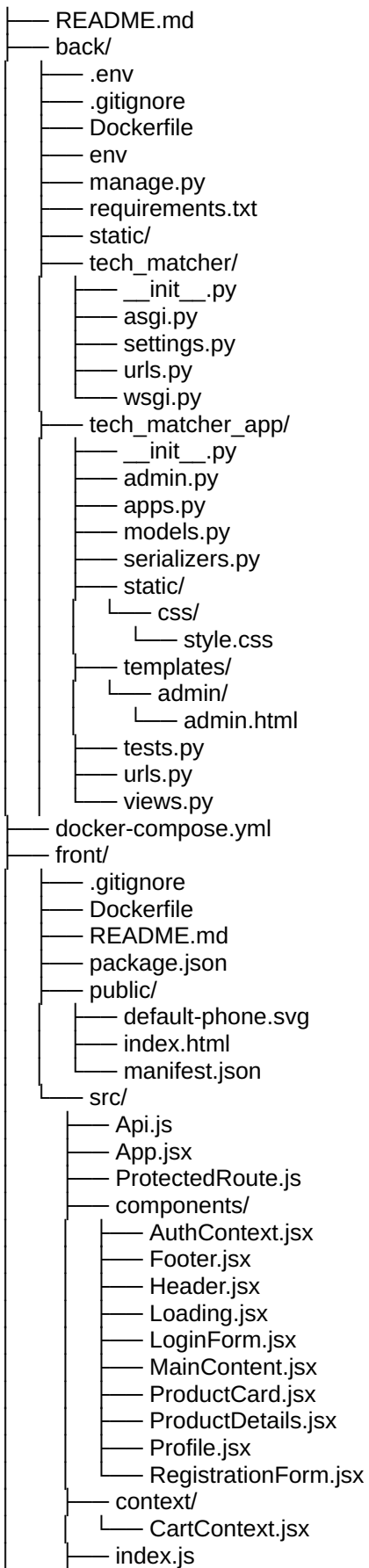
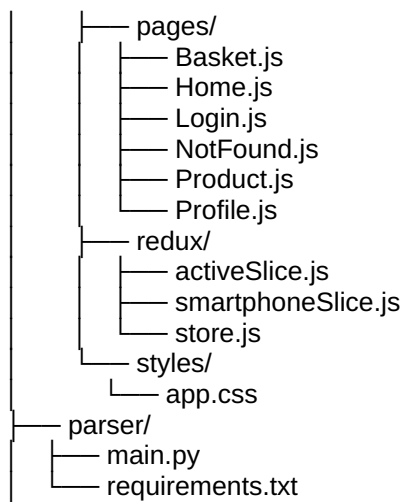


СТРУКТУРА ПРОЕКТА





ИСХОДНЫЙ КОД ФАЙЛОВ

=== ФАЙЛ: README.md ===

□ TechMatcher — Smartphone Selection Service

An intuitive web service designed to help users select smartphones based on technical specifications. The project is a Fullstack application featuring a decoupled architecture with an independent REST API (Backend) and an interactive user interface (Frontend).

□ Key Features

- * □ **Smart Search & Filtering:** Quick device selection based on core specifications and technologies.
- * □ **Detailed Specifications:** Visual and structured display of technical parameters for each smartphone.
- * □ **Order Placement:** Ability to submit a purchase request for the selected device.
- * □ **Automated Parsing:** Built-in module (`parser/`) to gather up-to-date data about smartphones.

□ Interface (Screenshots)

```
<div align="center">
  
  <p><i>Login screen of the application</i></p>

  
  <p><i>Main screen of the application and the device catalog</i></p>

  
  
  <p><i>Smartphone more details screen</i></p>
</div>
```

□ Tech Stack

- * **Backend:** Python, Django Rest Framework (DRF) — building the REST API architecture.
- * **Frontend:** JavaScript, React — creating a dynamic and responsive user interface.
- * **Database:** PostgreSQL (or any other compatible relational database).
- * **Parser:** Python scripts for automated database population.

□ Local Installation Guide

Prerequisites

- * Python 3.10+
- * Node.js (v18+)
- * PostgreSQL DBMS

Step 1: Clone the Repository

```
``bash
git clone https://github.com/Na1x-dev/TechMatcher
cd TechMatcher
``
```

Step 2: Backend Setup (Django DRF)

1. Navigate to the backend directory:

```
``bash
```

```

cd back
```
2. Create and activate a virtual environment:
```bash
python -m venv venv
# For Windows:
.\venv\Scripts\activate
# For macOS/Linux:
source venv/bin/activate
```
3. Install dependencies:
```bash
pip install -r requirements.txt
```
4. Configure the database connection in `settings.py` (or via `.env`) and run migrations:
```bash
python manage.py migrate
```
5. Start the local server:
```bash
python manage.py runserver
```
The backend API will be available at `http://127.0.0.1`

```

### ### Step 3: Frontend Setup (React)

```

1. Open a new terminal tab and navigate to the frontend folder from the project root:
```bash
cd front
```
2. Install NPM packages:
```bash
npm install
```
3. Start the client application:
```bash
npm start
# Or npm run dev (depending on your build tool)
```
The app will open at `http://localhost:3000`

```

---

### ## □ Project Structure

```

* `back` — Server-side source code (Django applications, API endpoints, serializers).
* `front` — Client-side code in React (components, styles, request logic).
* `parser` — Parsing module for smartphone data gathering.
* `md_imgs` — Static images for documentation.

```

---

### ## □ Contact

If you have any questions or suggestions for improving the project, feel free to contact me at: **na1x.1024@gmail.com**

### === ФАЙЛ: back/.env ===

```

SECRET_KEY=d1ddc36da27f46abf74bdd7e2c1e68d2defb1abb98b74dd50f41a3c2923762ef
POSTGRES_DB=ai_content_db
POSTGRES_USER=ai_user
POSTGRES_PASSWORD=ai_password
DATABASE_URL=postgres://ai_user:ai_password@db:5431/ai_content_db

```

### === ФАЙЛ: back/.gitignore ===

```

.env
media/images/

```

### === ФАЙЛ: back/Dockerfile ===

```

FROM python:3.11-slim

Устанавливаем рабочую директорию
WORKDIR /app

Запрещаем Python писать файлы .рус на диск и буферизовать stdout/stderr

```

```
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
```

```
Устанавливаем системные зависимости для psycopg2 (PostgreSQL)
RUN apt-get update && apt-get install -y --no-install-recommends \
 build-essential \
 libpq-dev \
 && rm -rf /var/list/apt/lists/*
```

```
Копируем и устанавливаем зависимости Python
COPY requirements.txt /app/
RUN pip install --no-cache-dir --upgrade pip && \
 pip install --no-cache-dir -r requirements.txt
```

```
Копируем исходный код бэкенда
COPY . /app/
```

```
Запускаем сервер Django на порту 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

=== ФАЙЛ: back/env ===

```
SECRET_KEY=d1ddc36da27f46abf74bdd7e2c1e68d2defb1abb98b74dd50f41a3c2923762ef
POSTGRES_DB=ai_content_db
POSTGRES_USER=ai_user
POSTGRES_PASSWORD=ai_password
DATABASE_URL=postgres://ai_user:ai_password@db:5431/ai_content_db
```

=== ФАЙЛ: back/manage.py ===

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
 """Run administrative tasks."""
 os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'tech_matcher.settings')
 try:
 from django.core.management import execute_from_command_line
 except ImportError as exc:
 raise ImportError(
 "Couldn't import Django. Are you sure it's installed and "
 "available on your PYTHONPATH environment variable? Did you "
 "forget to activate a virtual environment?"
) from exc
 execute_from_command_line(sys.argv)

if __name__ == '__main__':
 main()
```

=== ФАЙЛ: back/requirements.txt ===

```
asgiref==3.12.1
channels==4.3.2
Django==5.2.15
django-cors-headers==4.9.0
djangorestframework==3.18
djangorestframework-simplejwt==5.5.1
drf-yasg==1.21.15
inflection==0.5.1
packaging==26.3
pillow==12.3.0
psycopg2-binary==2.9.13
PyJWT==2.15.0
python-decouple==3.8
pytz==2026.4
PyYAML==6.0.3
sqlparse==0.6.0
uritemplate==4.2.0
```

=== ФАЙЛ: back/tech\_matcher/\_\_init\_\_.py ===

=== ФАЙЛ: back/tech\_matcher/asgi.py ===

"""

ASGI config for tech\_matcher project.

It exposes the ASGI callable as a module-level variable named ``application``.

For more information on this file, see

<https://docs.djangoproject.com/en/5.1/howto/deployment/asgi/>

"""

import os

from django.core.asgi import get\_asgi\_application

os.environ.setdefault('DJANGO\_SETTINGS\_MODULE', 'tech\_matcher.settings')

application = get\_asgi\_application()

=== ФАЙЛ: back/tech\_matcher/settings.py ===

from datetime import timedelta

from pathlib import Path

from decouple import config, Csv

import environ

import os

# Корневая директория проекта

BASE\_DIR = Path(\_\_file\_\_).resolve().parent.parent

env = environ.Env()

# Загрузка секретов из .env файла

SECRET\_KEY = env('SECRET\_KEY')

DEBUG = config('DEBUG', default=False, cast=bool)

ALLOWED\_HOSTS = config('ALLOWED\_HOSTS', default='127.0.0.1,localhost', cast=Csv())

# Настройки CORS

CORS\_ALLOWED\_ORIGINS = config(

'CORS\_ALLOWED\_ORIGINS',

default='http://localhost:3000',

cast=Csv()

)

# Приложения

INSTALLED\_APPS = [

'django.contrib.admin',

'django.contrib.auth',

'django.contrib.contenttypes',

'django.contrib.sessions',

'django.contrib.messages',

'django.contrib.staticfiles',

# Сторонние библиотеки

'rest\_framework',

'rest\_framework\_simplejwt',

'rest\_framework\_simplejwt.token\_blacklist',

'corsheaders',

'drf\_yasg',

'channels',

# Локальные приложения проекта

'tech\_matcher\_app.apps.TechMatcherAppConfig',

]

# Промежуточное ПО (Middleware)

MIDDLEWARE = [

'corsheaders.middleware.CorsMiddleware', # Должен быть как можно выше

'django.middleware.security.SecurityMiddleware',

'django.contrib.sessions.middleware.SessionMiddleware',

```

'django.middleware.common.CommonMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'tech_matcher.urls'

TEMPLATES = [
 {
 'BACKEND': 'django.template.backends.django.DjangoTemplates',
 'DIRS': [],
 'APP_DIRS': True,
 'OPTIONS': {
 'context_processors': [
 'django.template.context_processors.debug',
 'django.template.context_processors.request',
 'django.contrib.auth.context_processors.auth',
 'django.contrib.messages.context_processors.messages',
],
 },
],
]

WSGI_APPLICATION = 'tech_matcher.wsgi.application'
ASGI_APPLICATION = 'tech_matcher.asgi.application'

База данных (Конфигурируется через .env)
DATABASES = {
 'default': env.db('DATABASE_URL')
}

Кастомная модель пользователя
AUTH_USER_MODEL = 'tech_matcher_app.CustomUser'

Валидация паролей
AUTH_PASSWORD_VALIDATORS = [
 {'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
 {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
 {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
 {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

Интернационализация
LANGUAGE_CODE = 'ru-ru'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

Статические и медиа файлы
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / "static"]

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

Django REST Framework
REST_FRAMEWORK = {
 'DEFAULT_AUTHENTICATION_CLASSES': (
 'rest_framework_simplejwt.authentication.JWTAuthentication',
),
}

Настройки JWT (Безопасные тайминги)
SIMPLE_JWT = {
 'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30), # Короткий срок жизни для безопасности
 'REFRESH_TOKEN_LIFETIME': timedelta(days=7), # Срок жизни для обновления сессии
 'ROTATE_REFRESH_TOKENS': True,
 'BLACKLIST_AFTER_ROTATION': True,
 'AUTH_HEADER_TYPES': ('Bearer',),

```

```
}
```

=== ФАЙЛ: back/tech\_matcher/urls.py ===

```
from django.contrib import admin
from django.urls import path, include

from django.urls import path
from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from rest_framework import permissions

schema_view = get_schema_view(
 openapi.Info(
 title="Snippets API",
 default_version='v1',
 description="Test description",
 terms_of_service="https://www.google.com/policies/terms/",
 contact=openapi.Contact(email="contact@snippets.local"),
 license=openapi.License(name="BSD License"),
),
 validators=['ssv', 'flex'],
 public=True,
 permission_classes=(permissions.AllowAny,),
)

urlpatterns = [
 path('admin/', admin.site.urls),
 path('api/', include('tech_matcher_app.urls')),
 path('swagger/', schema_view.with_ui('swagger', cache_timeout=None), name='schema-swagger-ui'),
 path('redoc/', schema_view.with_ui('redoc', cache_timeout=None), name='schema-redoc'),
]
```

=== ФАЙЛ: back/tech\_matcher/wsgi.py ===

```
"""
```

WSGI config for tech\_matcher project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see

<https://docs.djangoproject.com/en/5.1/howto/deployment/wsgi/>

```
"""
```

```
import os
```

```
from django.core.wsgi import get_wsgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'tech_matcher.settings')
```

```
application = get_wsgi_application()
```

=== ФАЙЛ: back/tech\_matcher\_app/\_\_init\_\_.py ===

=== ФАЙЛ: back/tech\_matcher\_app/admin.py ===

```
from django.contrib import admin
from .models import CustomUser, Basket, BasketItem, Smartphone
from django.db import models
from django.forms import TextInput, Textarea
```

```
admin.site.register(CustomUser)
admin.site.register(Basket)
admin.site.register(BasketItem)
```

```
@admin.register(Smartphone)
class SmartphoneAdmin(admin.ModelAdmin):
 search_fields = ('title', 'brand')
 list_filter = ('brand', 'launch_year', 'os_version')
 ordering = ('launch_year',)
```

```
fieldsets = (
```

```

(None, {
 'fields': ('title', 'brand', 'launch_year', 'price')
}),
('Technical Specifications', {
 'fields': (
 'os_version', 'screen_size', 'ram_size', 'rom_size',
 'camera_count', 'main_camera_mp', 'cpu', 'gpu'
)
}),
('Dimensions and Materials', {
 'fields': (
 'length', 'width', 'thickness', 'weight',
 'back_material', 'edges_material'
)
}),
('Battery and Connectivity', {
 'fields': (
 'accum_volume', 'charging_power', 'wireless_charging',
 'bluetooth', 'wifi', '_5g'
)
}),
('Images and Media', {
 'fields': ('image_url',)
}),
)
formfield_overrides = {
 models.CharField: {'widget': TextInput(attrs={'size':20})},
 models.TextField: {'widget': Textarea(attrs={'rows':2, 'cols':100})},
}

```

=== ФАЙЛ: [back/tech\\_matcher\\_app/apps.py](#) ===

```
from django.apps import AppConfig
```

```

class TechMatcherAppConfig(AppConfig):
 default_auto_field = 'django.db.models.BigAutoField'
 name = 'tech_matcher_app'

```

=== ФАЙЛ: [back/tech\\_matcher\\_app/models.py](#) ===

```

from django.contrib.auth.models import AbstractBaseUser, BaseUserManager, PermissionsMixin
from django.db import models
from django.utils import timezone
from django.conf import settings

```

```

class CustomUserManager(BaseUserManager):
 def create_user(self, email, password=None, **extra_fields):
 if not email:
 raise ValueError('Электронная почта должна быть указана')
 email = self.normalize_email(email)
 user = self.model(email=email, **extra_fields)
 user.set_password(password)
 user.save(using=self._db)
 return user

```

```

 def create_superuser(self, email, password=None, **extra_fields):
 extra_fields.setdefault('is_staff', True)
 extra_fields.setdefault('is_superuser', True)
 return self.create_user(email, password, **extra_fields)

```

```

class CustomUser(AbstractBaseUser, PermissionsMixin):
 first_name = models.CharField(max_length=50, verbose_name='Имя')
 last_name = models.CharField(max_length=50, verbose_name='Фамилия')
 patronymic = models.CharField(max_length=50, verbose_name='Отчество', blank=True)
 email = models.EmailField(unique=True, verbose_name='Электронная почта')
 phone_number = models.CharField(max_length=20, verbose_name='Номер телефона', blank=True)
 image = models.ImageField(upload_to='images/%Y/%m/%d/', default='default_user.svg')

 is_active = models.BooleanField(default=True)
 is_staff = models.BooleanField(default=False)
 date_joined = models.DateTimeField(default=timezone.now)

```

```
USERNAME_FIELD = 'email'
REQUIRED_FIELDS = []
```

```
objects = CustomUserManager()
```

```
def __str__(self):
 if self.first_name and self.last_name:
 return f'{self.last_name} {self.first_name[0]}.'
 return f'admin - {self.email}'
```

```
class Meta:
 verbose_name = 'Пользователь'
 verbose_name_plural = 'Пользователи'
```

```
class Smartphone(models.Model):
 # Основная информация
 title = models.CharField(max_length=255, db_index=True)
 brand = models.CharField(max_length=100, db_index=True)
 launch_year = models.PositiveIntegerField(null=True, blank=True, db_index=True)
 price = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True, db_index=True)
 image_url = models.URLField(max_length=500, null=True, blank=True)

 # Технические характеристики
 os_version = models.CharField(max_length=100, blank=True)
 screen_size = models.FloatField(verbose_name="Диагональ экрана", null=True, blank=True)
 screen_res = models.CharField(max_length=50, blank=True)
 screen_type = models.CharField(max_length=100, blank=True)
 screen_fps = models.PositiveIntegerField(null=True, blank=True)
 screen_ratio = models.CharField(max_length=20, blank=True)
 screen_protector = models.CharField(max_length=100, blank=True)
 ppi = models.PositiveIntegerField(null=True, blank=True)
 color_count = models.CharField(max_length=50, blank=True)

 # Память
 ram_size = models.PositiveIntegerField(verbose_name="ОЗУ (ГБ)", null=True, blank=True)
 ram_type = models.CharField(max_length=50, blank=True)
 rom_size = models.PositiveIntegerField(verbose_name="ПЗУ (ГБ)", null=True, blank=True)
 rom_type = models.CharField(max_length=50, blank=True)

 # Камеры
 camera_count = models.PositiveIntegerField(default=1)
 main_camera_mp = models.CharField(max_length=100, blank=True)
 camera_type = models.CharField(max_length=255, blank=True)
 camera_block = models.TextField(blank=True)
 max_video_resolution = models.CharField(max_length=100, blank=True)
 front_camera_mp = models.CharField(max_length=50, blank=True)
 front_camera_aperture = models.CharField(max_length=50, blank=True)

 # Процессор
 cpu = models.CharField(max_length=255, blank=True)
 tech_process = models.CharField(max_length=50, blank=True)
 gpu = models.CharField(max_length=255, blank=True)

 # Корпус и материалы
 edges_material = models.CharField(max_length=100, blank=True)
 back_material = models.CharField(max_length=100, blank=True)
 back_color = models.CharField(max_length=100, blank=True)
 protection = models.CharField(max_length=100, blank=True)
 length = models.FloatField(null=True, blank=True)
 width = models.FloatField(null=True, blank=True)
 thickness = models.FloatField(null=True, blank=True)
 weight = models.FloatField(null=True, blank=True)

 # Аккумулятор
 accum_type = models.CharField(max_length=50, blank=True)
 accum_volume = models.PositiveIntegerField(verbose_name="Емкость батареи (mAh)", null=True, blank=True)
 charging_power = models.PositiveIntegerField(null=True, blank=True)
 wireless_charging = models.BooleanField(default=False)

 # Интерфейсы и связь
 bluetooth = models.CharField(max_length=50, blank=True)
 audio_port = models.CharField(max_length=50, blank=True)
```

```

charge_port = models.CharField(max_length=50, blank=True)
wifi = models.CharField(max_length=100, blank=True)
nfc = models.BooleanField(default=False)
has_5g = models.BooleanField(default=False)
sim_count = models.PositiveIntegerField(default=2)
sim_type = models.CharField(max_length=50, blank=True)

```

```

def __str__(self):
 return f'{self.brand} {self.title} ({self.launch_year})'

```

```

class BasketItem(models.Model):
 smartphone = models.ForeignKey(Smartphone, on_delete=models.CASCADE)
 quantity = models.PositiveIntegerField(default=1)

```

```

def __str__(self):
 return f'{self.smartphone.title} - {self.quantity} шт.'

```

```

class Basket(models.Model):
 user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
 items = models.ManyToManyField(BasketItem, blank=True)

```

```

def __str__(self):
 return f'Корзина пользователя: {self.user.email}'

```

```

def total_price(self):
 # Теперь это гарантированно работает, так как price — числовой тип
 return sum(
 (item.smartphone.price or 0) * item.quantity
 for item in self.items.all()
)

```

=== ФАЙЛ: [back/tech\\_matcher\\_app/serializers.py](#) ===

```

from rest_framework import serializers
from .models import CustomUser, Smartphone, Basket, BasketItem

```

# --- СЕРИАЛИЗАТОРЫ СМАРТФОНОВ ---

```

class SmartphoneSerializer(serializers.ModelSerializer):
 class Meta:
 model = Smartphone
 fields = '__all__'

```

# --- СЕРИАЛИЗАТОРЫ ПОЛЬЗОВАТЕЛЕЙ ---

```

class CustomUserSerializer(serializers.ModelSerializer):
 class Meta:
 model = CustomUser
 fields = ['id', 'email', 'first_name', 'last_name', 'patronymic', 'phone_number', 'image']
 read_only_fields = ['id', 'email']

```

```

class CustomUserCreateSerializer(serializers.ModelSerializer):
 password = serializers.CharField(write_only=True)

```

```

class Meta:
 model = CustomUser
 fields = ['email', 'first_name', 'last_name', 'patronymic', 'password', 'phone_number']

```

```

def create(self, validated_data):
 # Используем наш кастомный UserManager для правильного создания и хеширования
 return CustomUser.objects.create_user(**validated_data)

```

# --- СЕРИАЛИЗАТОРЫ КОРЗИНЫ ---

```

class BasketItemSerializer(serializers.ModelSerializer):
 # Включаем полную информацию о смартфоне прямо внутрь элемента корзины
 smartphone = SmartphoneSerializer(read_only=True)
 # Оставляем возможность передавать ID при добавлении/изменении
 smartphone_id = serializers.PrimaryKeyRelatedField(
 queryset=Smartphone.objects.all(),
 source='smartphone',
 write_only=True
)

```

```
)
```

```
class Meta:
 model = BasketItem
 fields = ['id', 'smartphone', 'smartphone_id', 'quantity']
```

```
class BasketSerializer(serializers.ModelSerializer):
 items = BasketItemSerializer(many=True, read_only=True)
 total_price = serializers.DecimalField(max_digits=10, decimal_places=2, read_only=True)
```

```
class Meta:
 model = Basket
 fields = ['id', 'user', 'items', 'total_price']
```

=== ФАЙЛ: back/tech\_matcher\_app/static/css/style.css ===

```
/* Изменение фона и цвета текста */
body {
 background-color: #F0A202; /* Основной фон */
 color: #5c374c; /* Цвет текста */
}

/* Заголовок админки */
h1, h2, h3, h4, h5, h6 {
 color: #ce6a85; /* Цвет заголовков */
}

/* Цвета кнопок */
button, .button {
 background-color: #f18805; /* Цвет кнопок */
 color: white; /* Цвет текста на кнопках */
}

/* Изменение цвета ссылок */
a {
 color: #985277; /* Цвет ссылок */
}

a:hover {
 color: #f18805; /* Цвет ссылок при наведении */
}

/* Изменение фона для форм */
form {
 background-color: #f0a202; /* Фон форм */
}

/* Изменение цвета полей ввода */
input[type="text"], input[type="number"], select {
 border: 1px solid #ce6a85; /* Цвет границы полей */
 background-color: #fff; /* Фон полей */
}
```

=== ФАЙЛ: back/tech\_matcher\_app/templates/admin/admin.html ===

```
{% extends "admin/admin.html" %}

{% block extrastyle %}
 {{ block.super }}
 <link rel="stylesheet" type="text/css" href="{% static '/css/style.css' %}">
{% endblock %}
```

=== ФАЙЛ: back/tech\_matcher\_app/tests.py ===

```
from django.test import TestCase
```

```
Create your tests here.
```

=== ФАЙЛ: back/tech\_matcher\_app/urls.py ===

```
from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
```

```

from .views import (
 RegisterView,
 UserProfileAPIView,
 SmartphoneListView,
 SmartphoneDetailView,
 BasketView
)

urlpatterns = [
 # Авторизация и токены (JWT)
 path('token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
 path('token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),

 # Пользователи
 path('register/', RegisterView.as_view(), name='register'),
 path('users/<int:id>/', UserProfileAPIView.as_view(), name='user-profile'),

 # Каталог смартфонов
 path('smartphones/', SmartphoneListView.as_view(), name='smartphone-list'),
 path('smartphones/<int:id>/', SmartphoneDetailView.as_view(), name='smartphone-detail'),

 # Корзина
 path('basket/', BasketView.as_view(), name='user-basket'),
]

```

=== ФАЙЛ: back/tech\_matcher\_app/views.py ===

```

from rest_framework import generics, status
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework.pagination import PageNumberPagination
from rest_framework.parsers import MultiPartParser, FormParser

from .models import CustomUser, Smartphone, Basket, BasketItem
from .serializers import (
 CustomUserSerializer,
 CustomUserCreateSerializer,
 SmartphoneSerializer,
 BasketSerializer
)

--- ПАГИНАЦИЯ ---
class SmartphonePagination(PageNumberPagination):
 page_size = 24
 page_size_query_param = 'page_size'
 max_page_size = 100

--- ВЬЮШКИ СМАРТФОНОВ ---
class SmartphoneListView(generics.ListAPIView):
 """
 Получение списка всех смартфонов с поддержкой пагинации.
 Сюда же позже очень легко добавятся фильтры.
 """
 queryset = Smartphone.objects.all().order_by('-launch_year')
 serializer_class = SmartphoneSerializer
 pagination_class = SmartphonePagination
 permission_classes = [AllowAny]

class SmartphoneDetailView(generics.RetrieveAPIView):
 """
 Получение детальной информации об одном смартфоне по ID.
 """
 queryset = Smartphone.objects.all()
 serializer_class = SmartphoneSerializer
 permission_classes = [AllowAny]
 lookup_field = 'id'

--- ВЬЮШКИ ПОЛЬЗОВАТЕЛЕЙ И АВТОРИЗАЦИИ ---
class RegisterView(generics.CreateAPIView):
 """

```

```

Регистрация нового пользователя.
"""
queryset = CustomUser.objects.all()
serializer_class = CustomUserCreateSerializer
permission_classes = [AllowAny]

class UserProfileAPIView(generics.RetrieveUpdateAPIView):
 """
 Просмотр и обновление профиля текущего пользователя.
 Использует MultiPartParser для чистой и безопасной загрузки картинок.
 """
 queryset = CustomUser.objects.all()
 serializer_class = CustomUserSerializer
 permission_classes = [IsAuthenticated]
 parser_classes = [MultiPartParser, FormParser]
 lookup_field = 'id'

 def get_object(self):
 # Гарантируем, что пользователь может редактировать ТОЛЬКО свой профиль
 return self.request.user

--- ВЬЮШКА КОРЗИНЫ ---
class BasketView(APIView):
 """
 Управление корзиной пользователя (Получение, добавление, удаление элементов).
 """
 permission_classes = [IsAuthenticated]

 def get(self, request):
 basket, _ = Basket.objects.get_or_create(user=request.user)
 serializer = BasketSerializer(basket)
 return Response(serializer.data, status=status.HTTP_200_OK)

 def post(self, request):
 basket, _ = Basket.objects.get_or_create(user=request.user)
 smartphone_id = request.data.get('smartphone_id')
 quantity = int(request.data.get('quantity', 1))

 if not smartphone_id:
 return Response({"detail": "smartphone_id обязателен"}, status=status.HTTP_400_BAD_REQUEST)

 try:
 smartphone = Smartphone.objects.get(id=smartphone_id)
 except Smartphone.DoesNotExist:
 return Response({"detail": "Смартфон не найден"}, status=status.HTTP_404_NOT_FOUND)

 # Ищем, есть ли уже такой товар в корзине
 basket_item = basket.items.filter(smartphone=smartphone).first()
 if basket_item:
 basket_item.quantity += quantity
 basket_item.save()
 else:
 basket_item = BasketItem.objects.create(smartphone=smartphone, quantity=quantity)
 basket.items.add(basket_item)

 return Response(BasketSerializer(basket).data, status=status.HTTP_201_CREATED)

 def delete(self, request):
 basket, _ = Basket.objects.get_or_create(user=request.user)
 smartphone_id = request.data.get('smartphone_id')

 try:
 item = basket.items.get(smartphone_id=smartphone_id)
 basket.items.remove(item)
 item.delete()
 return Response(BasketSerializer(basket).data, status=status.HTTP_200_OK)
 except BasketItem.DoesNotExist:
 return Response({"detail": "Товар в корзине не найден"}, status=status.HTTP_404_NOT_FOUND)

```

=== ФАЙЛ: docker-compose.yml ===  
version: '3.8'

```

services:
База данных PostgreSQL
db:
 image: postgres:15-alpine
 container_name: tech_matcher_db
 restart: always
 volumes:
 - postgres_data:/var/lib/postgresql/data/
 environment:
 POSTGRES_DB: tech_matcher
 POSTGRES_USER: postgres
 POSTGRES_PASSWORD: postgres
 ports:
 - "5431:5431"
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U postgres -d tech_matcher"]
 interval: 5s
 timeout: 5s
 retries: 5

Бэкенд на Django REST Framework
backend:
 build: ./back
 container_name: tech_matcher_backend
 restart: always
 command: >
 sh -c "python manage.py makemigrations &&
 python manage.py migrate &&
 python manage.py init &&
 python manage.py runserver 0.0.0.0:8000"
 volumes:
 - ./back:/app
 ports:
 - "8000:8000"
 depends_on:
 db:
 condition: service_healthy

Фронтенд на React
frontend:
 build: ./front
 container_name: tech_matcher_frontend
 restart: always
 volumes:
 - ./front:/app
 - /app/node_modules
 ports:
 - "3000:3000"
 environment:
 - CHOKIDAR_USEPOLLING=true # Обеспечивает hot-reload внутри контейнера Docker
 depends_on:
 - backend
 stdin_open: true # Исправляет баг, когда контейнер React сразу завершает работу
 tty: true

volumes:
 postgres_data:

```

=== ФАЙЛ: front/.gitignore ===

```

node_modules
/build
coverage
.env.local
.env.development.local
.env.test.local
.env.production.local
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.DS_Store

```

### === ФАЙЛ: front/Dockerfile ===

```
FROM node:18-slim
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

### === ФАЙЛ: front/README.md ===

# TechMatcher — обновлённый frontend

Полностью обновлённый интерфейс React-приложения с сохранением текущих API-контрактов:

- `POST /api/token/` — вход
- `POST /api/register/` — регистрация
- `GET /api/smartphones/?page=&search=` — каталог
- `GET /api/smartphones/:id/` — карточка товара
- `GET /api/basket/` — серверная корзина
- `POST /api/basket/` — добавление товара
- `DELETE /api/basket/?smartphone\_id=` — удаление товара
- `GET/PUT /api/users/:id/` — профиль

## Запуск

```
```bash
npm install
npm start
```
```

По умолчанию API: `http://localhost:8000/api`.  
Для другого адреса можно задать:

```
```bash
REACT_APP_API_URL=http://your-host/api npm start
```
```

## Корзина

Гость: товары сохраняются в `localStorage` под ключом `techmatcher\_guest\_cart`.

После входа локальные товары отправляются в `POST /basket/`, после чего корзина продолжает работать через сервер.

Оформление заказа намеренно не имитирует успешный заказ: в исходном проекте отдельного API-контракта заказа не было, поэтому кнопка показывает информационное сообщение.

### === ФАЙЛ: front/package.json ===

```
{
 "name": "tech_matcher",
 "version": "1.0.0",
 "private": true,
 "dependencies": {
 "@emotion/react": "^11.13.3",
 "@emotion/styled": "^11.13.0",
 "@mui/material": "^6.1.1",
 "@reduxjs/toolkit": "^2.2.7",
 "@testing-library/jest-dom": "^5.17.0",
 "@testing-library/react": "^13.4.0",
 "@testing-library/user-event": "^13.5.0",
 "axios": "^1.7.7",
 "notistack": "^3.0.1",
 "react": "^18.3.1",
 "react-dom": "^18.3.1",
 "react-redux": "^9.1.2",
 "react-router-dom": "^6.26.1",
 "react-scripts": "^5.0.1",
 "web-vitals": "^2.1.4"
 },
 "scripts": {
 "start": "react-scripts start",
 "build": "react-scripts build",
 "test": "react-scripts test",
 "eject": "react-scripts eject"
 }
}
```

```

},
"eslintConfig": { "extends": ["react-app", "react-app/jest"] },
"browserslist": {
 "production": [">0.2%", "not dead", "not op_mini all"],
 "development": ["last 1 chrome version", "last 1 firefox version", "last 1 safari version"]
}
}
}

```

### === ФАЙЛ: front/public/default-phone.svg ===

```

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 300 420"><rect width="300" height="420" rx="36" fill="#eef0f4"/><rect x="28"
y="18" width="244" height="384" rx="30" fill="#151a22"/><rect x="43" y="55" width="214" height="315" rx="12" fill="#f7f8fa"/><circle
cx="150" cy="37" r="4" fill="#727b8a"/><text x="150" y="225" text-anchor="middle" font-family="Arial" font-size="28" font-weight="700"
fill="#a4abb7">TECH</text><text x="150" y="255" text-anchor="middle" font-family="Arial" font-size="18" fill="#c1c6cf">MATCHER</
text></svg>

```

### === ФАЙЛ: front/public/index.html ===

```

<!DOCTYPE html>
<html lang="ru">
<head>
 <meta charset="utf-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <meta name="theme-color" content="#111827" />
 <meta name="description" content="TechMatcher — магазин смартфонов и гаджетов" />
 <title>TechMatcher</title>
</head>
<body>
 <noscript>Для работы приложения необходимо включить JavaScript.</noscript>
 <div id="root"></div>
</body>
</html>

```

### === ФАЙЛ: front/public/manifest.json ===

```

{
 "short_name": "TechMatcher",
 "name": "TechMatcher",
 "start_url": ".",
 "display": "standalone",
 "theme_color": "#111827",
 "background_color": "#f5f7fb"
}

```

### === ФАЙЛ: front/src/Api.js ===

```

import axios from "axios";

export const baseUrl = process.env.REACT_APP_API_URL || "http://localhost:8000/api";

const apiClient = axios.create({
 baseUrl: baseUrl.replace(/\/$/, ""),
 headers: { "Content-Type": "application/json" },
});

apiClient.interceptors.request.use((config) => {
 const token = localStorage.getItem("accessToken");
 if (token) config.headers.Authorization = `Bearer ${token}`;
 else delete config.headers.Authorization;
 return config;
});

export const getReq = async (endpoint, config = {}) => (await apiClient.get(endpoint, config)).data;
export const postReq = async (endpoint, data, config = {}) => (await apiClient.post(endpoint, data, config)).data;
export const putReq = async (endpoint, data, config = {}) => (await apiClient.put(endpoint, data, config)).data;
export const deleteReq = async (endpoint, config = {}) => (await apiClient.delete(endpoint, config)).data;

export const getApiError = (error, fallback = "Произошла ошибка. Попробуйте ещё раз.") => {
 const data = error?.response?.data;
 if (typeof data === "string" && data) return data;
 if (data?.detail) return data.detail;
 if (data && typeof data === "object") {
 const first = Object.values(data).flat?.()[0];

```

```

 if (first) return String(first);
 }
 return fallback;
};

```

```
export default apiClient;
```

### === ФАЙЛ: front/src/App.jsx ===

```

import React from "react";
import { Navigate, Route, Routes } from "react-router-dom";
import Home from "../pages/Home";
import Login from "../pages/Login";
import Basket from "../pages/Basket";
import Product from "../pages/Product";
import Profile from "../pages/Profile";
import NotFound from "../pages/NotFound";
import ProtectedRoute from "../ProtectedRoute";

const App = () => <Routes>
 <Route path="/" element={<Home />} />
 <Route path="/login" element={<Login />} />
 <Route path="/basket" element={<Basket />} />
 <Route path="/smartphones/:id" element={<Product />} />
 <Route path="/profile" element={<ProtectedRoute><Profile /></ProtectedRoute>} />
 <Route path="/headphones" element={<Navigate to="/" replace />} />
 <Route path="/fitness-bracelets" element={<Navigate to="/" replace />} />
 <Route path="/chargers" element={<Navigate to="/" replace />} />
 <Route path="*" element={<NotFound />} />
</Routes>;
export default App;

```

### === ФАЙЛ: front/src/ProtectedRoute.js ===

```

import React from "react";
import { Navigate, useLocation } from "react-router-dom";
import { useAuth } from "../components/AuthContext";
const ProtectedRoute = ({ children }) => { const { isAuthenticated } = useAuth(); const location = useLocation(); return
isAuthenticated() ? children : <Navigate to="/login" replace state={{ from: location.pathname }} />; };
export default ProtectedRoute;

```

### === ФАЙЛ: front/src/components/AuthContext.jsx ===

```

import React, { createContext, useContext, useMemo, useState } from "react";

const AuthContext = createContext(null);

const parseJwt = (token) => {
 if (!token) return null;
 try {
 const part = token.split(".")[1];
 if (!part) return null;
 const base64 = part.replace(/-/g, "+").replace(/_/g, "");
 return JSON.parse(decodeURIComponent(atob(base64).split("").map((c) => `%${('00${c.charCodeAt(0).toString(16)})'.slice(-2)}`
).join("")));
 } catch {
 return null;
 }
};

export const AuthProvider = ({ children }) => {
 const [user, setUser] = useState(() => parseJwt(localStorage.getItem("accessToken")));

 const login = (accessToken, refreshToken) => {
 localStorage.setItem("accessToken", accessToken);
 if (refreshToken) localStorage.setItem("refreshToken", refreshToken);
 setUser(parseJwt(accessToken));
 };

 const logout = () => {
 localStorage.removeItem("accessToken");
 localStorage.removeItem("refreshToken");
 setUser(null);
 };

```

```

const value = useMemo(() => ({ user, login, logout, isAuthenticated: () => Boolean(user) }), [user]);
return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

export const useAuth = () => {
 const context = useContext(AuthContext);
 if (!context) throw new Error("useAuth должен использоваться внутри AuthProvider");
 return context;
};

```

### === ФАЙЛ: front/src/components/Footer.jsx ===

```

import React from "react";
const Footer = () => <footer className="site-footer"><div>TechMatcherМагазин техники с понятным выбором.© {new Date().getFullYear()}</div></footer>;
export default Footer;

```

### === ФАЙЛ: front/src/components/Header.jsx ===

```

import React, { useEffect, useState } from "react";
import { NavLink, useNavigate } from "react-router-dom";
import { getReq } from "../Api";
import { useAuth } from "../AuthContext";
import { useCart } from "../context/CartContext";

const Header = () => {
 const navigate = useNavigate();
 const { user, logout } = useAuth();
 const { itemCount } = useCart();
 const [profile, setProfile] = useState(null);
 const [menuOpen, setMenuOpen] = useState(false);

 useEffect(() => {
 if (!user?.user_id) { setProfile(null); return; }
 getReq(`users/${user.user_id}/`).then(setProfile).catch(() => setProfile(null));
 }, [user]);

 const displayName = profile?.first_name
 ? `${profile.first_name}${profile.last_name ? ` ${profile.last_name[0]}.` : ""}`
 : user?.email || "Пользователь";

 const doLogout = () => { setMenuOpen(false); logout(); navigate("/"); };

 return (
 <header className="site-header">
 <div className="header-inner">
 <button className="brand" onClick={() => navigate("/") } aria-label="TechMatcher">
 TTechMatcher
 </button>
 <nav className="main-nav">
 <NavLink to="/" end>Смартфоны</NavLink>
 { e.preventDefault(); document.getElementById("categories")?.scrollIntoView({ behavior: "smooth" }); }}>Категории
 </nav>
 <div className="header-actions">
 <button className="icon-button cart-button" onClick={() => navigate("/basket")} aria-label="Корзина">
 🛒{itemCount > 0 && {itemCount}}</button>
 {user ? (
 <div className="profile-menu-wrap">
 <button className="profile-trigger" onClick={() => setMenuOpen((v) => !v)}>
 {displayName[0]?.toUpperCase()}
 {displayName}👤
 </button>
 {menuOpen && <div>
 <button className="menu-backdrop" aria-label="Закрыть меню" onClick={() => setMenuOpen(false)} />
 <div className="profile-menu">
 <button onClick={() => { navigate("/profile"); setMenuOpen(false); }}>Профиль</button>
 <button onClick={() => { navigate("/basket"); setMenuOpen(false); }}>Корзина</button>
 <button onClick={doLogout}>Выйти</button>
 </div>
 </div>
)}
 </div>
 </div>
 </div>
 </header>
);
}

```

```

) : <button className="button button-dark" onClick={() => navigate("/login")}>Войти</button>
 </div>
</div>
</header>
);
};
export default Header;

```

### === ФАЙЛ: front/src/components/Loading.jsx ===

```

import React from "react";

const Loading = ({ label = "Загрузка..." }) => (
 <div className="state-screen" role="status">
 <div className="spinner" />
 {label}
 </div>
);
export default Loading;

```

### === ФАЙЛ: front/src/components/LoginForm.jsx ===

```

import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useSnackbar } from "notistack";
import { postReq, getApiError } from "../Api";
import { useAuth } from "../AuthContext";

const LoginForm = ({ onToggleRegister }) => {
 const navigate = useNavigate(); const { enqueueSnackbar } = useSnackbar(); const { login } = useAuth();
 const [form, setForm] = useState({ email: "", password: "" }); const [loading, setLoading] = useState(false);
 const submit = async (e) => { e.preventDefault(); if (!form.email || !form.password) return enqueueSnackbar("Заполните email и пароль", { variant: "warning" }); setLoading(true); try { const data = await postReq("token/", form); if (!data?.access) throw new Error("No token"); login(data.access, data.refresh); enqueueSnackbar("Добро пожаловать!", { variant: "success" }); navigate("/"); } catch (err) { enqueueSnackbar(getApiError(err, "Неверный email или пароль"), { variant: "error" }); } finally { setLoading(false); } };
 return <form className="auth-form" onSubmit={submit}><div className="auth-title">WELCOME BACK<h1>С возвращением.</h1><p>Войдите, чтобы синхронизировать корзину и открыть профиль.</p></div><label className="field">Email<input type="email" autoComplete="email" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} placeholder="you@example.com" /></label><label className="field">Пароль<input type="password" autoComplete="current-password" value={form.password} onChange={(e) => setForm({ ...form, password: e.target.value })} placeholder="....." /></label><button className="button button-primary button-large" disabled={loading}>{loading ? "Входим..." : "Войти"}</button><button type="button" className="auth-switch" onClick={onToggleRegister}>Ещё нет аккаунта? Зарегистрироваться</button></form>;
};
export default LoginForm;

```

### === ФАЙЛ: front/src/components/MainContent.jsx ===

```

import React, { useCallback, useEffect, useState } from "react";
import { getReq, getApiError } from "../Api";
import { useSnackbar } from "notistack";
import ProductCard from "../ProductCard";
import Loading from "../Loading";

const MainContent = () => {
 const { enqueueSnackbar } = useSnackbar();
 const [products, setProducts] = useState([]); const [loading, setLoading] = useState(true);
 const [search, setSearch] = useState(""); const [query, setQuery] = useState(""); const [page, setPage] = useState(1); const [totalPages, setTotalPages] = useState(1);
 useEffect(() => { const t = setTimeout(() => { setQuery(search.trim()); setPage(1); }, 450); return () => clearTimeout(t); }, [search]);
 const load = useCallback(async () => {
 setLoading(true);
 try {
 const params = new URLSearchParams({ page: String(page) }); if (query) params.set("search", query);
 const data = await getReq(`smartphones/${params}`); const list = Array.isArray(data) ? data : (data.results || []);
 setProducts(list); setTotalPages(data.count ? Math.max(1, Math.ceil(data.count / 24)) : (data.next || data.previous ? page + (data.next ? 1 : 0) : 1));
 } catch (error) { enqueueSnackbar(getApiError(error, "Не удалось загрузить каталог"), { variant: "error" }); }
 finally { setLoading(false); }
 }, [page, query, enqueueSnackbar]);
 useEffect(() => { load(); }, [load]);
 return <main>
 <section className="hero"><div><div className="hero-kicker">TECHMATCHER / CATALOG</div><h1>Техника, которую
легко выбрать.</h1><p>Сравнивайте характеристики смартфонов и собирайте корзину без лишних шагов.</p></div>

```

```


<div className="hero-orb">01<small>SMART
CHOICE</small></div></section>
 <section className="catalog-section" id="categories">
 <div className="section-head"><div>КАТАЛОГ<h2>Смартфоны</h2></div><div
className="search-box">🔍<input value={search} onChange={(e) => setSearch(e.target.value)} placeholder="Поиск
по модели или бренду..." /></div></div>
 {loading ? <Loading /> : products.length ? <><div className="product-grid">{products.map((p) => <ProductCard key={p.id}
product={p} />)}</div>{totalPages > 1 && <div className="pagination"><button disabled={page === 1} onClick={() => setPage((p) =>
p - 1)}>← Назад</button>{page} / {totalPages}<button disabled={page >= totalPages} onClick={() => setPage((p) => p
+ 1)}>Вперёд →</button></div></> : <div className="empty-state"><div>📱</div><h3>Ничего не нашли</h3><p>Попробуйте
изменить поисковый запрос.</p></div>}
 </section>
</main>;
};
export default MainContent;


```

### === ФАЙЛ: front/src/components/ProductCard.jsx ===

```

import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useSnackbar } from "notistack";
import { useCart } from "../context/CartContext";

const money = (value) => value == null || value === "" ? "Цена не указана" : `${Number(value).toLocaleString("ru-RU")} ₽`;

const ProductCard = ({ product }) => {
 const navigate = useNavigate();
 const { enqueueSnackbar } = useSnackbar();
 const { addToCart } = useCart();
 const [adding, setAdding] = useState(false);
 const add = async (e) => {
 e.stopPropagation(); setAdding(true);
 try { await addToCart(product); enqueueSnackbar("Товар добавлен в корзину", { variant: "success" }); }
 catch { enqueueSnackbar("Не удалось добавить товар", { variant: "error" }); }
 finally { setAdding(false); }
 };
 return <article className="product-card" onClick={() => navigate(`/smartphones/${product.id}`)}>
 <div className="product-image-wrap"></div>
 <div className="product-card-body">
 <div className="eyebrow">{product.brand || "Смартфон"}</div>
 <h3>{product.title}</h3>
 <div className="product-tags">
 {product.launch_year && {product.launch_year} r.}
 {product.ram_size && product.rom_size && {product.ram_size} Гб}{product.rom_size} Гб</div>
 <div className="product-card-bottom">{money(product.price)}<button className="button button-primary"
onClick={add} disabled={adding}>{adding ? "...": "В корзину"}</button></div>
 </div>
 </article>;
};
export default ProductCard;

```

### === ФАЙЛ: front/src/components/ProductDetails.jsx ===

```

import React, { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { getApiError, getReq } from "../Api";
import { useSnackbar } from "notistack";
import { useCart } from "../context/CartContext";
import Loading from "../Loading";

const money = (value) => value == null || value === "" ? "Цена не указана" : `${Number(value).toLocaleString("ru-RU")} ₽`;
const groups = [
 ["Основное", [{"Год релиза", "launch_year"}, ["Операционная система", "os_version"], ["Материал задней панели",
"back_material"], ["Материал граней", "edges_material"], ["Цвет", "back_color"], ["Класс защиты", "protection"]]],
 ["Дисплей", [{"Диагональ", "screen_size", ""}, ["Разрешение", "screen_res"], ["Тип матрицы", "screen_type"], ["Частота
обновления", "screen_fps", " Гц"], ["Плотность пикселей", "ppi", " PPI"]]],
 ["Процессор и память", [{"Процессор", "cpu"}, ["Техпроцесс", "tech_process"], ["Графика", "gpu"], ["ОЗУ", "ram_size", " ГБ"], ["Тип
ОЗУ", "ram_type"], ["Накопитель", "rom_size", " ГБ"], ["Тип накопителя", "rom_type"]]],
 ["Батарея и интерфейсы", [{"Аккумулятор", "accum_volume", " мАч"}, ["Зарядка", "charging_power", " Вт"], ["Беспроводная
зарядка", "wireless_charging"], ["NFC", "nfc"], ["5G", "has_5g"], ["Порт зарядки", "charge_port"], ["Аудиопорт", "audio_port"]]],
];
const displayValue = (v, suffix = "") => typeof v === "boolean" ? (v ? "Есть" : "Нет") : `${v}${suffix}`;

```

```

const ProductDetails = () => {
 const { id } = useParams(); const navigate = useNavigate(); const { enqueueSnackbar } = useSnackbar(); const { addToCart } = useCart();
 const [product, setProduct] = useState(null); const [loading, setLoading] = useState(true); const [adding, setAdding] = useState(false);
 useEffect(() => { setLoading(true); getReq(`smartphones/${id}/`).then(setProduct).catch((e) => enqueueSnackbar(getApiError(e, "Смартфон не найден"), { variant: "error" })).finally(() => setLoading(false)); }, [id, enqueueSnackbar]);
 if (loading) return <Loading />;
 if (!product) return <div className="state-screen"><h2>Смартфон не найден</h2><button className="button button-primary" onClick={() => navigate("/")}>В каталог</button></div>;
 const add = async () => { setAdding(true); try { await addToCart(product); enqueueSnackbar("Товар добавлен в корзину", { variant: "success" }); } catch (e) { enqueueSnackbar(getApiError(e, "Не удалось добавить товар"), { variant: "error" }); } finally { setAdding(false); } };
 return <main className="product-page"><button className="back-link" onClick={() => navigate(-1)}>← Назад</button><div className="product-layout">
 <div className="detail-visual"><div className="detail-image"></div><div className="detail-note">TECHMATCHER
ORIGINAL CATALOG</div></div>
 <div className="detail-copy"><div className="eyebrow">>{product.brand || "SMARTPHONE"}</div><h1>{product.title}</h1><p className="detail-price">{money(product.price)}</p><p className="detail-intro">Подробные характеристики устройства собраны в одном месте — от экрана и процессора до интерфейсов и аккумулятора.</p><button className="button button-primary button-large" onClick={add} disabled={adding}>{adding ? "Добавляем..." : "Добавить в корзину"}</button></div>
 </div><section className="specs"><div className="section-kicker">TECH SPECS</div><h2>Характеристики</h2><div className="spec-grid">{groups.map(([{title}, rows]) => <div className="spec-card" key={title}><h3>{title}</h3><rows.map(([{label, key, suffix = ""}] => product[key] !== undefined && product[key] !== null && product[key] !== "" ? <div className="spec-row" key={key}>{label}{displayValue(product[key], suffix)}</div> : null)}</div>)}</div></div></div></div>
);
};
export default ProductDetails;

```

### === ФАЙЛ: front/src/components/Profile.jsx ===

```

import React, { useEffect, useState } from "react";
import { getApiError, getReq, putReq } from "../Api";
import { useAuth } from "../AuthContext";
import { useSnackbar } from "notistack";
import Loading from "../Loading";

const Profile = () => {
 const { user, logout } = useAuth(); const { enqueueSnackbar } = useSnackbar();
 const [profile, setProfile] = useState({ first_name: "", last_name: "", patronymic: "", phone_number: "", image: null }); const [preview, setPreview] = useState(""); const [loading, setLoading] = useState(true); const [saving, setSaving] = useState(false);
 useEffect(() => { if (!user?.user_id) return; getReq(`users/${user.user_id}/`).then((d) => { setProfile({ first_name: d.first_name || "", last_name: d.last_name || "", patronymic: d.patronymic || "", phone_number: d.phone_number || "", image: null }); setPreview(d.image || ""); }).catch((e) => enqueueSnackbar(getApiError(e, "Не удалось загрузить профиль"), { variant: "error" })).finally(() => setLoading(false)); }, [user, enqueueSnackbar]);
 if (loading) return <Loading />;
 const submit = async (e) => { e.preventDefault(); setSaving(true); const data = new FormData(); Object.entries(profile).forEach(([key, value]) => { if (key !== "image") data.append(key, value); }); if (profile.image) data.append("image", profile.image); try { const updated = await putReq(`users/${user.user_id}/`, data, { headers: { "Content-Type": "multipart/form-data" } }); if (updated.image) setPreview(updated.image); enqueueSnackbar("Профиль сохранён", { variant: "success" }); } catch (err) { enqueueSnackbar(getApiError(err, "Не удалось сохранить профиль"), { variant: "error" }); } finally { setSaving(false); } };
 return <main className="account-page"><div className="account-head"><div><div className="section-kicker">ACCOUNT</div><h1>Личный кабинет</h1></div><button className="button button-ghost" onClick={logout}>Выйти</button></div><form className="profile-card" onSubmit={submit}><div className="profile-aside"><div className="big-avatar">{preview ? : (profile.first_name?.[0] || user.email?.[0] || "U").toUpperCase()}</div><label className="button button-outline">Изменить фото<input type="file" accept="image/*" hidden onChange={(e) => { const file = e.target.files?.[0]; if (file) { setProfile((p) => ({ ...p, image: file }); }); setPreview(URL.createObjectURL(file)); } } /></label>JPG, PNG или WEBP</div><div className="form-grid">{[["last_name", "Фамилия", true], ["first_name", "Имя", true], ["patronymic", "Отчество", false], ["phone_number", "Телефон", false]].map(([{name, label, required}] => <label className="field" key={name}>{label}<input name={name} value={profile[name]} required={required} onChange={(e) => setProfile((p) => ({ ...p, [name]: e.target.value }))) /></label>)}</div><div className="form-actions"><button className="button button-primary" disabled={!saving}>{saving ? "Сохраняем..." : "Сохранить изменения"}</button></div></div></form></main>;
};
export default Profile;

```

### === ФАЙЛ: front/src/components/RegistrationForm.jsx ===

```

import React, { useState } from "react";
import { useSnackbar } from "notistack";
import { getApiError, postReq } from "../Api";

const RegistrationForm = ({ onToggleLogin }) => {
 const { enqueueSnackbar } = useSnackbar(); const [loading, setLoading] = useState(false); const [form, setForm] = useState({ email: "", first_name: "", last_name: "", patronymic: "", password: "", phone_number: "" });

```

```

const submit = async (e) => { e.preventDefault(); if (!form.email || !form.first_name || !form.last_name || !form.password) return
enqueueSnackbar("Заполните обязательные поля", { variant: "warning" }); setLoading(true); try { await postReq("register/", form);
enqueueSnackbar("Аккаунт создан. Теперь войдите.", { variant: "success" }); onToggleLogin(); } catch (err)
{ enqueueSnackbar(getApiError(err, "Не удалось зарегистрироваться"), { variant: "error" }); } finally { setLoading(false); } };
return <form className="auth-form" onSubmit={submit}><div className="auth-title">CREATE
ACCOUNT<h1>Создайте аккаунт.</h1><p>Ваши данные и корзина будут доступны после входа с любого сеанса.</p></
div>[[["email", "Email", "email", true], ["last_name", "Фамилия", "text", true], ["first_name", "Имя", "text", true],
["patronymic", "Отчество", "text", false], ["phone_number", "Телефон", "tel", false],
["password", "Пароль", "password", true]].map(([{name, label, type, required}]) => <label className="field" key={name}>{label}</
span><input type={type} required={required} value={form[name]} onChange={(e) => setForm({ ...form, [name]: e.target.value })} /></
label>)><button className="button button-primary button-large" disabled={loading}>{loading ? "Создаём..." : "Зарегистрироваться"}
</button><button type="button" className="auth-switch" onClick={onToggleLogin}>Уже есть аккаунт? Войти</
button></form>;
};
export default RegistrationForm;

```

### === ФАЙЛ: front/src/context/CartContext.jsx ===

```

import React, { createContext, useCallbck, useContext, useEffect, useMemo, useRef, useState } from "react";
import { deleteReq, getReq, postReq } from "../Api";
import { useAuth } from "../components/AuthContext";

```

```

const STORAGE_KEY = "techmatcher_guest_cart";
const CartContext = createContext(null);

```

```

const readGuestCart = () => {
 try { return JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]"); } catch { return []; }
};
const writeGuestCart = (items) => localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
const countItems = (items) => items.reduce((sum, item) => sum + Number(item.quantity || 0), 0);

```

```

export const CartProvider = ({ children }) => {
 const { user } = useAuth();
 const [guestItems, setGuestItems] = useState(readGuestCart());
 const [serverBasket, setServerBasket] = useState(null);
 const [loading, setLoading] = useState(false);
 const previousUser = useRef(user);

```

```

 const refreshServerCart = useCallbck(async () => {
 if (!user) { setServerBasket(null); return null; }
 setLoading(true);
 try {
 const basket = await getReq("basket/");
 setServerBasket(basket);
 return basket;
 } finally { setLoading(false); }
 }, [user]);

```

```

 const syncGuestCart = useCallbck(async () => {
 const local = readGuestCart();
 if (!user || !local.length) return;
 setLoading(true);
 try {
 for (const item of local) {
 await postReq("basket/", { smartphone_id: item.id, quantity: item.quantity });
 }
 localStorage.removeItem(STORAGE_KEY);
 setGuestItems([]);
 await refreshServerCart();
 } finally { setLoading(false); }
 }, [refreshServerCart, user]);

```

```

 useEffect(() => {
 const becameAuthenticated = !previousUser.current && user;
 previousUser.current = user;
 if (user) {
 if (becameAuthenticated && readGuestCart().length) syncGuestCart().catch(() => {});
 else refreshServerCart().catch(() => {});
 } else {
 setServerBasket(null);
 setGuestItems(readGuestCart());
 }
 }, [user, refreshServerCart, syncGuestCart]);

```

```

const addToCart = async (product, quantity = 1) => {
 if (!user) {
 const current = readGuestCart();
 const index = current.findIndex((item) => item.id === product.id);
 if (index >= 0) current[index] = { ...current[index], quantity: current[index].quantity + quantity };
 else current.push({ id: product.id, quantity, product });
 writeGuestCart(current); setGuestItems(current); return;
 }
 await postReq("basket/", { smartphone_id: product.id, quantity });
 await refreshServerCart();
};

const removeFromCart = async (id) => {
 if (!user) {
 const next = readGuestCart().filter((item) => item.id !== id);
 writeGuestCart(next); setGuestItems(next); return;
 }
 const updated = await deleteReq(`basket/?smartphone_id=${id}`);
 setServerBasket(updated);
};

const items = user ? (serverBasket?.items || []) : guestItems;
const total = user
 ? Number(serverBasket?.total_price || 0)
 : guestItems.reduce((sum, item) => sum + Number(item.product?.price || 0) * Number(item.quantity || 0), 0);
const itemCount = countItems(items);

const value = useMemo(() => ({ items, total, itemCount, loading, addToCart, removeFromCart, refresh: refreshServerCart, isGuest: !
user }), [items, total, itemCount, loading, user, refreshServerCart]);
return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
};

export const useCart = () => {
 const context = useContext(CartContext);
 if (!context) throw new Error("useCart должен использоваться внутри CartProvider");
 return context;
};

```

### === ФАЙЛ: front/src/index.js ===

```

import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { Provider } from "react-redux";
import { SnackbarProvider } from "notistack";
import store from "../redux/store";
import { AuthProvider } from "../components/AuthContext";
import { CartProvider } from "../context/CartContext";
import App from "../App";
import "../styles/app.css";

ReactDOM.createRoot(document.getElementById("root")).render(<React.StrictMode><Provider store={store}>
 <BrowserRouter><SnackbarProvider maxSnack={3} autoHideDuration={3000} anchorOrigin={{ vertical: "bottom", horizontal: "right" }}>
 <AuthProvider><CartProvider><App /></CartProvider></AuthProvider></SnackbarProvider></BrowserRouter></Provider></
 React.StrictMode>);

```

### === ФАЙЛ: front/src/pages/Basket.js ===

```

import React from "react";
import { useNavigate } from "react-router-dom";
import { useSnackbar } from "notistack";
import { useCart } from "../context/CartContext";
import Loading from "../components/Loading";

const money = (v) => `${Number(v || 0).toLocaleString("ru-RU")} Р`;
const Basket = () => {
 const navigate = useNavigate(); const { enqueueSnackbar } = useSnackbar(); const { items, total, itemCount, loading,
 removeFromCart, isGuest } = useCart();
 if (loading) return <Loading />;
 return <main className="cart-page"><div className="cart-head"><div className="section-kicker">YOUR CART</
 div><h1>Корзина</h1></div>{itemCount} {itemCount === 1 ? "товар" : "товаров"}</div>
 {isGuest && items.length > 0 && <div className="guest-banner">Корзина сохранена на этом устройстве.</
 strong>Войдите позже — товары синхронизируются с вашим аккаунтом.<button onClick={() => navigate("/login")}>

```

```

>Войти</button></div>
 {items.length ? <div className="empty-state cart-empty"><div>□ </div><h2>Корзина пока пустая</h2><p>Добавьте смартфоны
из каталога — они появятся здесь.</p><button className="button button-primary" onClick={() => navigate("/")}>Перейти в
каталог</button></div> : <div className="cart-layout"><div className="cart-list">{items.map((item) => { const p = item.smartphone
|| item.product; return <article className="cart-item" key={item.id || p.id}><button className="cart-product" onClick={() => navigate(
smartphones/${p.id})}><div className="cart-image"></div><div><div
className="eyebrow">{p.brand}</div><h3>{p.title}</h3>Количество: {item.quantity}</div></div></button><div
className="cart-item-side">{money(Number(p.price) * Number(item.quantity))}<button className="remove"
onClick={() => removeFromCart(p.id).catch(() => enqueueSnackbar("Не удалось удалить товар", { variant: "error" })}>Удалить</
button></div></article>}}</div><aside className="cart-summary">ORDER SUMMARY</
span><h2>Итого</h2><div className="summary-row">Товары{money(total)}</div><div
className="summary-row">ДоставкаПо условиям заказа</div><div className="summary-
total">K оплате{money(total)}</div><button className="button button-primary button-large"
onClick={() => enqueueSnackbar("Оформление заказа пока не подключено к API бэкенда", { variant: "info" })}>Оформить заказ</
button></aside></div>
</main>;
};
export default Basket;

```

### === ФАЙЛ: front/src/pages/Home.js ===

```

import React from "react";
import Header from "../components/Header";
import MainContent from "../components/MainContent";
import Footer from "../components/Footer";
const Home = () => <div className="app-shell"><Header /><MainContent /><Footer /></div>;
export default Home;

```

### === ФАЙЛ: front/src/pages/Login.js ===

```

import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import LoginForm from "../components/LoginForm";
import RegistrationForm from "../components/RegistrationForm";

const Login = () => { const [register, setRegister] = useState(false); const navigate = useNavigate(); return <main className="auth-
page"><div className="auth-brand"><button className="brand" onClick={() => navigate("/")}>T</
span>TechMatcher</button><div className="auth-poster">SMARTCHOICE</
strong><small>find your device</small></div></div><div className="auth-panel">{register ? <RegistrationForm onToggleLogin={() =>
setRegister(false)} /> : <LoginForm onToggleRegister={() => setRegister(true)} />}</div></main>; };
export default Login;

```

### === ФАЙЛ: front/src/pages/NotFound.js ===

```

import React from "react";
import { useNavigate } from "react-router-dom";
import Header from "../components/Header";
import Footer from "../components/Footer";
const NotFound = () => { const navigate = useNavigate(); return <div className="app-shell"><Header /><main className="not-
found">404<h1>Страница не найдена</h1><p>Похоже, такой страницы в TechMatcher больше нет.</p><button
className="button button-primary" onClick={() => navigate("/")}>Вернуться в каталог</button></main><Footer /></div>; };
export default NotFound;

```

### === ФАЙЛ: front/src/pages/Product.js ===

```

import React from "react";
import Header from "../components/Header";
import Footer from "../components/Footer";
import ProductDetails from "../components/ProductDetails";
const Product = () => <div className="app-shell"><Header /><ProductDetails /><Footer /></div>;
export default Product;

```

### === ФАЙЛ: front/src/pages/Profile.js ===

```

import React from "react";
import Header from "../components/Header";
import Footer from "../components/Footer";
import Profile from "../components/Profile";
const ProfilePage = () => <div className="app-shell"><Header /><Profile /><Footer /></div>;
export default ProfilePage;

```

### === ФАЙЛ: front/src/redux/activeSlice.js ===

```
import { createSlice } from "@reduxjs/toolkit";
const slice = createSlice({ name: "active", initialState: { isActive: false }, reducers: { toggleActive: (state) => { state.isActive = !state.isActive; }, setActive: (state, action) => { state.isActive = action.payload; } } });
export const { toggleActive, setActive } = slice.actions; export default slice.reducer;
```

### === ФАЙЛ: front/src/redux/smartphoneSlice.js ===

```
import { createSlice } from "@reduxjs/toolkit";
const slice = createSlice({ name: "smartphone", initialState: { smartphone: null }, reducers: { setSmartphone: (state, action) => { state.smartphone = action.payload; } } });
export const { setSmartphone } = slice.actions; export default slice.reducer;
```

### === ФАЙЛ: front/src/redux/store.js ===

```
import { configureStore, createSlice } from "@reduxjs/toolkit";
import activeReducer from "./activeSlice";
import smartphoneReducer from "./smartphoneSlice";
const loginFormSlice = createSlice({ name: "loginForm", initialState: { isVisible: false }, reducers: { showLoginForm: (s) => { s.isVisible = true; }, hideLoginForm: (s) => { s.isVisible = false; } } });
export const { showLoginForm, hideLoginForm } = loginFormSlice.actions;
export default configureStore({ reducer: { active: activeReducer, smartphone: smartphoneReducer, loginForm: loginFormSlice.reducer } });
```

### === ФАЙЛ: front/src/styles/app.css ===

```
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Space+Grotesk:wght@500;600;700&display=swap');
:root{--ink:#111827;--muted:#697386;--line:#e5e8ef;--bg:#f5f7fb;--card:#fff;--accent:#ff5b35;--accent2:#ff784e;--dark:#151a22;--radius:22px;--shadow:0 18px 50px rgba(17,24,39,.08)}*{box-sizing:border-box}html{scroll-behavior:smooth}
body{margin:0;background:var(--bg);color:var(--ink);font-family:'DM Sans',system-ui,sans-serif;-webkit-font-smoothing:antialiased}
button,input{font:inherit}button{cursor:pointer}button:disabled{cursor:not-allowed;opacity:.5}a{text-decoration:none;color:inherit}.app-shell{min-height:100vh;display:flex;flex-direction:column}.site-header{height:78px;background:rgba(245,247,251,.9);backdrop-filter:blur(18px);border-bottom:1px solid rgba(229,232,239,.8);position:sticky;top:0;z-index:20}.header-inner{max-width:1240px;height:100%;margin:auto;padding:0 24px;display:flex;align-items:center;gap:42px}.brand{border:0;background:none;display:flex;align-items:center;gap:10px;font:700 20px 'Space Grotesk';color:var(--ink);padding:0}.brand-mark{width:34px;height:34px;border-radius:11px;background:var(--ink);color:#fff;display:grid;place-items:center}.main-nav{display:flex;gap:26px;font-weight:600;color:#7b8494}.main-nav a{position:relative;padding:8px 0}.main-nav a.active,.main-nav a:hover{color:var(--ink)}.main-nav a.active:after{content:"";position:absolute;left:0;right:0;bottom:0;height:2px;background:var(--accent)}.header-actions{margin-left:auto;display:flex;align-items:center;gap:10px}.icon-button,.profile-trigger{border:1px solid var(--line);background:#fff;border-radius:13px;height:44px}.icon-button{width:44px;position:relative}.icon-button b{position:absolute;right:-5px;top:-6px;background:var(--accent);color:#fff;border-radius:99px;min-width:19px;height:19px;font-size:11px;display:grid;place-items:center}.profile-trigger{display:flex;align-items:center;gap:9px;padding:5px 10px 5px 5px;font-weight:600}.avatar,.big-avatar{display:grid;place-items:center;border-radius:50%;background:#e6eff4;color:var(--ink);font-weight:700}.avatar{width:32px;height:32px}.profile-menu-wrap{position:relative}.profile-menu{position:absolute;right:0;top:52px;width:190px;background:#fff;border:1px solid var(--line);box-shadow:var(--shadow);border-radius:16px;padding:7px;z-index:30}.profile-menu button{width:100%;text-align:left;border:0;background:none;border-radius:10px;padding:11px 12px}.profile-menu button:hover{background:#f4f5f7}.menu-backdrop{position:fixed;inset:0;background:transparent;border:0;z-index:-1}.button{border:0;border-radius:13px;padding:11px 18px;font-weight:700;transition:.18s ease}.button:hover{transform:translateY(-1px)}.button-primary{background:var(--accent);color:#fff;box-shadow:0 10px 22px rgba(255,91,53,.2)}.button-primary:hover{background:#f34d27}.button-dark{background:var(--dark);color:#fff}.button-outline{background:#fff;border:1px solid var(--line);color:var(--ink)}.button-ghost{background:#fff;border:1px solid var(--line)}.button-large{width:100%;padding:15px 20px;font-size:16px}
.hero{max-width:1240px;margin:0 auto;padding:92px 24px 86px;display:grid;grid-template-columns:1.2fr .8fr;align-items:center;min-height:450px}.hero-kicker,.section-kicker,.eyebrow{font-size:11px;letter-spacing:.14em;font-weight:800;color:#8a93a3}.hero h1{font:700 clamp(50px,7vw,88px)/.92 'Space Grotesk';letter-spacing:-.06em;margin:17px 0 26px}.hero h1 em{font-style:normal;color:var(--accent)}.hero p{max-width:530px;font-size:18px;line-height:1.65;color:var(--muted);margin:0}.hero-orb{width:280px;height:280px;border-radius:50%;background:var(--dark);color:#fff;justify-self:end;display:flex;flex-direction:column;justify-content:center;padding:52px;transform:rotate(8deg);box-shadow:25px 30px 0 #e9ecf2}.hero-orb span{font:700 72px 'Space Grotesk';color:var(--accent)}.hero-orb small{font-weight:700;letter-spacing:.18em;line-height:1.5}.catalog-section{max-width:1240px;margin:auto;padding:0 24px 90px;flex:1}.section-head{display:flex;justify-content:space-between;align-items:end;gap:24px;margin-bottom:30px}.section-head h2,.specs h2,.account-head h1,.cart-head h1{font:700 42px 'Space Grotesk';letter-spacing:-.04em;margin:7px 0 0}.search-box{width:min(410px,100%);height:52px;background:#fff;border:1px solid var(--line);border-radius:15px;display:flex;align-items:center;padding:0 16px;gap:10px}.search-box span{font-size:26px;color:#9aa2af}.search-box input{border:0;outline:0;width:100%;background:transparent;color:var(--ink)}.product-grid{display:grid;grid-template-columns:repeat(4,1fr);gap:16px}.product-card{background:var(--card);border:1px solid var(--line);border-radius:var(--radius);overflow:hidden;cursor:pointer;transition:.22s ease}.product-card:hover{transform:translateY(-5px);box-shadow:var(--shadow);border-color:#dce0e7}.product-image-wrap{height:260px;background:#f7f8fa;display:grid;place-items:center;padding:28px}.product-image-wrap img{max-width:100%;max-height:210px;width:100%;height:100%;object-fit:contain}.product-card-body{padding:18px}.product-card h3{font:600 18px 'Space Grotesk';margin:7px 0 13px;white-space:nowrap;overflow:hidden;text-overflow:ellipsis}.product-tags{display:flex;gap:7px;min-height:28px}.product-tags span{background:#f0f2f5;color:#737d8c;border-radius:8px;padding:5px 8px;font-size:11px}.product-card-bottom{display:flex;align-items:center;justify-content:space-between;gap:8px;margin-top:19px}.product-card-bottom strong{font:700 19px 'Space Grotesk'}.product-card-bottom .button{padding:9px 11px;font-size:12px}.pagination{display:flex;justify-content:center;align-items:center;gap:18px;margin-top:34px}.pagination button{border:1px solid var(--line);background:#fff;border-radius:11px;padding:10px
```

15px}.pagination span{font-weight:700;color:#697386}.empty-state{text-align:center;background:#fff;border:1px dashed #d7dce5;border-radius:24px;padding:65px 20px;color:var(--muted)}.empty-state>div{font-size:40px}.empty-state h3,.empty-state h2{color:var(--ink);margin:12px 0 5px}.state-screen{min-height:55vh;display:flex;align-items:center;justify-content:center;gap:14px;flex-direction:column;color:var(--muted)}.spinner{width:34px;height:34px;border:3px solid #e2e5eb;border-top-color:var(--accent);border-radius:50%;animation:spin .8s linear infinite}@keyframes spin{to{transform:rotate(360deg)}}.site-footer{margin-top:auto;background:var(--dark);color:#fff}.site-footer>div{max-width:1240px;margin:auto;padding:28px 24px;display:flex;gap:20px;align-items:center}.site-footer strong{font:700 17px 'Space Grotesk'}.site-footer span{color:#a9b0bd;font-size:13px}.site-footer span:last-child{margin-left:auto}

.product-page,.account-page,.cart-page{max-width:1240px;margin:auto;width:100%;padding:42px 24px 90px}.back-link{border:0;background:none;padding:0;color:#747e8f;font-weight:700;margin-bottom:30px}.product-layout{display:grid;grid-template-columns:1fr 1fr;gap:70px;align-items:center}.detail-visual{position:relative}.detail-image{height:520px;background:#fff;border:1px solid var(--line);border-radius:30px;display:grid;place-items:center;padding:55px}.detail-image img{max-width:100%;max-height:420px;object-fit:contain}.detail-note{position:absolute;left:-15px;bottom:-15px;background:var(--dark);color:#fff;padding:15px 18px;border-radius:13px;font:700 11px 'Space Grotesk';letter-spacing:.13em;line-height:1.6}.detail-note span{color:#aeb5c1;font-size:8px}.detail-copy h1{font:700 clamp(42px,5vw,68px)/.98 'Space Grotesk';letter-spacing:-.055em;margin:10px 0 20px}.detail-price{font:700 30px 'Space Grotesk';margin:0 0 20px}.detail-intro{color:var(--muted);font-size:17px;line-height:1.7;max-width:500px;margin-bottom:30px}.detail-copy .button{max-width:320px}.specs{margin-top:90px}.spec-grid{display:grid;grid-template-columns:repeat(2,1fr);gap:16px;margin-top:25px}.spec-card{background:#fff;border:1px solid var(--line);border-radius:20px;padding:24px}.spec-card h3{font:600 21px 'Space Grotesk';margin:0 0 17px}.spec-row{display:flex;justify-content:space-between;gap:20px;padding:12px 0;border-top:1px solid #edf0f4;font-size:14px}.spec-row span{color:#788294}.spec-row strong{text-align:right}.account-head,.cart-head{display:flex;justify-content:space-between;align-items:end;margin-bottom:30px}.profile-card{background:#fff;border:1px solid var(--line);border-radius:26px;padding:34px;display:grid;grid-template-columns:250px 1fr;gap:45px}.profile-aside{display:flex;align-items:center;flex-direction:column;border-right:1px solid var(--line);padding-right:40px}.big-avatar{width:150px;height:150px;font-size:52px;overflow:hidden;margin-bottom:18px}.big-avatar img{width:100%;height:100%;object-fit:cover}.profile-aside .button{position:relative}.profile-aside .muted{font-size:11px;color:#98a0ad;margin-top:9px}.form-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px}.field{display:flex;flex-direction:column;gap:8px}.field span{font-size:12px;font-weight:700;color:#727c8b}.field input{height:48px;border:1px solid var(--line);border-radius:12px;padding:0 14px;outline:0;background:#fafbfc}.field input:focus{border-color:#b9bec8;box-shadow:0 0 3px #0f1f3}.form-actions{grid-column:1/-1;display:flex;justify-content:flex-end;margin-top:5px}.cart-head>span{color:#778193;font-weight:700}.cart-layout{display:grid;grid-template-columns:1.5fr .7fr;gap:20px;align-items:start}.cart-list{display:flex;flex-direction:column;gap:12px}.cart-item{background:#fff;border:1px solid var(--line);border-radius:20px;padding:16px;display:flex;justify-content:space-between;gap:20px}.cart-product{display:flex;gap:17px;align-items:center;border:0;background:none;text-align:left;padding:0;min-width:0}.cart-image{width:105px;height:105px;background:#f6f7f9;border-radius:15px;display:grid;place-items:center;flex:none}.cart-image img{max-width:80%;max-height:85%;object-fit:contain}.cart-product h3{font:600 19px 'Space Grotesk';margin:6px 0}.cart-product span{font-size:13px;color:#7a8493}.cart-item-side{display:flex;flex-direction:column;align-items:end;justify-content:space-between;padding:4px 0}.cart-item-side strong{font:700 19px 'Space Grotesk'}.remove{border:0;background:none;color:#e2554b;font-size:12px;font-weight:700}.cart-summary{background:var(--dark);color:#fff;border-radius:24px;padding:27px;position:sticky;top:105px}.cart-summary h2{font:600 30px 'Space Grotesk';margin:8px 0 28px}.summary-row,.summary-total{display:flex;justify-content:space-between;gap:10px;padding:12px 0;color:#aeb6c3;font-size:14px}.summary-row strong{color:#fff}.summary-total{border-top:1px solid #343b47;margin-top:8px;padding-top:20px;color:#fff;font-size:17px}.summary-total strong{font-size:21px}.cart-summary .button{margin-top:20px}.guest-banner{background:#fff4ef;border:1px solid #ffd9cd;border-radius:16px;padding:14px 17px;margin-bottom:17px;display:flex;gap:12px;align-items:center;font-size:13px}.guest-banner strong{color:#b74429}.guest-banner span{color:#7b625a}.guest-banner button{margin-left:auto;border:0;background:none;color:#bd462d;font-weight:800}.not-found{flex:1;min-height:65vh;display:flex;flex-direction:column;align-items:center;justify-content:center;text-align:center}.not-found>span{font:700 150px/8 'Space Grotesk';color:#e7eaf0}.not-found h1{font:700 38px 'Space Grotesk';margin:30px 0 8px}.not-found p{color:var(--muted);margin:0 0 25px}

.auth-page{min-height:100vh;display:grid;grid-template-columns:1fr 520px;background:#fff}.auth-brand{background:var(--dark);color:#fff;padding:32px 5vw;position:relative;overflow:hidden}.auth-brand .brand{color:#fff}.auth-brand .brand-mark{background:var(--accent)}.auth-poster{position:absolute;left:9%;bottom:12%;display:flex;flex-direction:column;font:700 100px/78 'Space Grotesk';letter-spacing:-.07em}.auth-poster span{color:#8d96a6}.auth-poster strong{color:#fff}.auth-poster small{font:600 13px 'DM Sans';letter-spacing:.2em;color:var(--accent);margin-top:28px}.auth-panel{display:flex;align-items:center;padding:50px 60px}.auth-form{width:100%;max-width:410px;margin:auto}.auth-title{margin-bottom:30px}.auth-title h1{font:700 42px 'Space Grotesk';letter-spacing:-.05em;margin:8px 0 10px}.auth-title p{color:var(--muted);line-height:1.6;margin:0}.auth-form .field{margin-bottom:18px}.auth-switch{display:block;border:0;background:none;margin:18px auto 0;color:#7b8492;font-size:13px}.auth-switch strong{color:var(--accent)}

@media(max-width:1050px){.product-grid{grid-template-columns:repeat(3,1fr)}.hero{padding-top:65px}.profile-name{display:none}.product-layout{gap:35px}}@media(max-width:760px){.site-header{height:auto}.header-inner{padding:13px 16px;gap:14px;flex-wrap:wrap}.main-nav{order:3;width:100%;overflow:auto;padding-bottom:3px;gap:20px;font-size:13px}.hero{grid-template-columns:1fr;padding:55px 18px 60px}.hero h1{font-size:56px}.hero-orb{display:none}.catalog-section,.product-page,.account-page,.cart-page{padding-left:16px;padding-right:16px}.section-head{align-items:stretch;flex-direction:column}.section-head h2,.specs h2,.account-head h1,.cart-head h1{font-size:34px}.product-grid{grid-template-columns:repeat(2,1fr)}.product-image-wrap{height:200px;padding:20px}.product-card-body{padding:13px}.product-card h3{font-size:15px}.product-card-bottom{align-items:stretch;flex-direction:column}.product-card-bottom .button{width:100%}.product-layout,.cart-layout{grid-template-columns:1fr}.detail-image{height:380px}.spec-grid{grid-template-columns:1fr}.profile-card{grid-template-columns:1fr;padding:22px}.profile-aside{border-right:0;border-bottom:1px solid var(--line);padding:0 0 25px}.form-grid{grid-template-columns:1fr}.form-actions{grid-column:auto}.cart-summary{position:static}.guest-banner{align-items:flex-start;flex-direction:column}.guest-banner button{margin:0}.auth-page{grid-template-columns:1fr}.auth-brand{min-height:170px;padding:22px}.auth-poster{display:none}.auth-panel{padding:35px 22px}.site-footer>div{flex-wrap:wrap}.site-footer span:last-child{margin-left:0;width:100%}}@media(max-width:460px){.product-grid{grid-template-columns:1fr}.profile-trigger{padding-right:7px}.cart-item{align-items:flex-start}.cart-product{align-items:flex-start}.cart-image{width:80px;height:80px}.cart-item-side strong{font-size:15px}.cart-item-side{gap:20px}.hero h1{font-size:47px}}

[illegible]

```

Строгое приведение типов перед отправкой в базу данных
cleaned_values = (
 str(data.get('title', 'Unknown')),
 str(data.get('brand', 'Unknown')),
 clean_int(data.get('launch_year')),
 clean_decimal(data.get('price')),
 str(data.get('image_url', "")) if data.get('image_url') else None,

 str(data.get('os_version', "")),
 clean_float(data.get('screen_size')),
 str(data.get('screen_res', "")),
 str(data.get('screen_type', "")),
 clean_int(data.get('screen_fps')),

 str(data.get('screen_ratio', "")),
 str(data.get('screen_protector', "")),
 clean_int(data.get('ppi')),
 str(data.get('color_count', "")),

 clean_int(data.get('ram_size')),
 str(data.get('ram_type', "")),
 clean_int(data.get('rom_size')),
 str(data.get('rom_type', "")),

 clean_int(data.get('camera_count')) or 1,
 str(data.get('main_camera_mp', "")),
 str(data.get('camera_type', "")),
 str(data.get('camera_block', "")),

 str(data.get('max_video_resolution', "")),
 str(data.get('front_camera_mp', "")),
 str(data.get('front_camera_aperture', "")),

 str(data.get('cpu', "")),
 str(data.get('tech_process', "")),
 str(data.get('gpu', "")),

 str(data.get('edges_material', "")),
 str(data.get('back_material', "")),
 str(data.get('back_color', "")),
 str(data.get('protection', "")),
 clean_float(data.get('length')),
 clean_float(data.get('width')),
 clean_float(data.get('thickness')),
 clean_float(data.get('weight')),

 str(data.get('accum_type', "")),
 clean_int(data.get('accum_volume')),
 clean_int(data.get('charging_power')),
 clean_bool(data.get('wireless_charging')),

 str(data.get('bluetooth', "")),
 str(data.get('audio_port', "")),
 str(data.get('charge_port', "")),
 str(data.get('wifi', "")),
 clean_bool(data.get('nfc')),
 clean_bool(data.get('has_5g')),
 clean_int(data.get('sim_count')) or 2,
 str(data.get('sim_type', ""))
)

cursor.execute(insert_query, cleaned_values)
connection.commit()

except Exception as error:
 print(f"Ошибка при записи смартфона в базу данных: {error}")
finally:
 if connection:
 cursor.close()
 connection.close()

Пример запуска / тестирования функции
if __name__ == "__main__":

```

```

Тестовый сырой объект, имитирующий результат парсинга сайта (например, с характеристиками в виде строк)
raw_parsed_data = {
 "title": "iPhone 15 Pro",
 "brand": "Apple",
 "launch_year": "2023 г.",
 "price": "120 000 руб.",
 "screen_size": "6.1\"",
 "screen_fps": "120 Гц",
 "ram_size": "8 ГБ",
 "rom_size": "256 ГБ",
 "accum_volume": "3274 мАч",
 "wireless_charging": "Есть",
 "nfc": "+",
 "has_5g": "Да"
}

print("Запуск тестового сохранения...")
save_smartphone_to_db(raw_parsed_data)

```

=== ФАЙЛ: parser/requirements.txt ===

```

aptdaemon==2.0.2
attrs==25.4.0
autocommand==2.2.2
babel==2.17.0
bcc==0.35.0
bcrypt==5.0.0
blinker==1.9.0
Brlapi==0.8.6
certifi==2026.1.4
chardet==5.2.0
click==8.1.8
command-not-found==0.3
configobj==5.0.9
cryptography==46.0.5
cupshelpers==1.0
dasbus==1.7
dbus-python==1.4.0
defer==1.0.6
distro==1.9.0
distro-info==1.15
gyp-next==0.16.2
httplib2==0.22.0
idna==3.11
inflect==7.5.0
jaraco.context==6.0.1
jaraco.functools==4.1.0
jaraco.text==4.0.0
Jinja2==3.1.6
jsonpatch==1.32
jsonpointer==2.4
jsonschema==4.19.2
jsonschema-specifications==2023.12.1
language-selector==0.1
launchpadlib==2.1.0
lazr.restfulclient==0.14.6
lazr.uri==1.0.6
libpass==1.9.3
linkify-it-py==2.0.3
louis==3.36.0
markdown-it-py==3.0.0
MarkupSafe==3.0.3
mdurl==0.1.2
more-itertools==10.8.0
netaddr==1.3.0
oauthlib==3.3.1
olefile==0.47
packaging==26.0
pexpect==4.9.0
pillow==12.1.1
psutil==7.1.0
ptyprocess==0.7.0
pycairo==1.27.0
pycup==2.0.4

```

Pygments==2.19.2  
PyGObject==3.56.2  
PyJWT==2.10.1  
pyparsing==3.3.2  
pyserial==3.5  
python-apt==3.1.0+ubuntu1  
python-debian==1.0.1+ubuntu2  
pyxdg==0.28  
PyYAML==6.0.3  
referencing==0.36.2  
requests==2.32.5  
rich==13.9.4  
rpds-py==0.27.1  
setproctitle==1.3.7  
setuptools==78.1.1  
ssh-import-id==5.11  
systemd-python==235  
typeguard==4.4.4  
typing\_extensions==4.15.0  
ubuntu-drivers-common==0.0.0  
ubuntu-pro-client==8001  
uc-micro-py==1.0.3  
ufw==0.36.2  
unattended-upgrades==0.1  
urllib3==2.6.3  
wadllib==2.0.0  
wheel==0.46.3  
xdg==5  
xkit==0.0.0  
zipp==3.23.0